

Machine Learning Regression Analysis

CE 475 Project Report

Prediction of Target Variable Y
from Six Independent Features

Demir Cücü
20220602018

20 May 2026

1. Methodology

This project addresses a supervised regression problem: predicting a continuous target variable Y from six independent features (X_1 through X_6). The training dataset comprises 100 labeled samples, while the test set contains 20 unlabeled samples (ID's 101-120) for which Y values must be predicted.

A multi model comparison strategy was adopted to identify the most accurate regression approach. Eight distinct regression algorithms were evaluated, spanning linear, instance based, kernel based, and ensemble methods. This breadth of coverage ensures that the best performing paradigm is captured regardless of the underlying data structure.

1.1. Models Attempted

The following eight regression models were selected for evaluation. The choice was driven by the need to test fundamentally different learning paradigms:

Model	Type	Pros	Cons
Ridge Regression	Linear	Stable with multicollinearity, L2 regularization	Cannot capture non-linearity
Lasso Regression	Linear	Feature selection via L1 sparsity	May underfit complex data
ElasticNet	Linear	Combines L1+L2 penalties	Requires tuning of two parameters
Huber Regression	Linear (Robust)	Resistant to outliers in Y	Assumes approximate linearity
K-Nearest Neighbors	Instance-Based	No training phase, flexible	Sensitive to scale, no feature importance
Random Forest	Ensemble (Bagging)	Low variance, robust to outliers	Can overfit small datasets
Gradient Boosting ★	Ensemble (Boosting)	Sequential error correction, high accuracy	Hyperparameter-sensitive
XGBoost	Ensemble (Boosting)	Regularized boosting, efficient	Complex hyperparameter space

★ Best model, selected via 108 configuration grid search. SVR (RBF Kernel) was excluded: combined with the symlog target transform it produced numerically degenerate CV results ($MAE > 21,000$; $R^2 = -113,673$) and is therefore omitted from all comparisons.

1.2. Rationale for Model Selection

An initial exploratory data analysis revealed two critical characteristics of the dataset:

- **Weak linear correlations:** All features exhibit absolute Pearson correlation coefficients below 0.35 with Y (X_1 : $r = -0.35$, X_5 : $r = +0.24$), confirming that simple linear models are insufficient.
- **Perfect multicollinearity between X_3 and X_6 :** Analysis revealed that $X_6 = 3 \cdot X_3 + 2$ ($r = 1.0000$), meaning the dataset effectively contains only five independent features. This redundancy destabilizes linear model estimation but does not affect tree-based models.
- **Highly skewed target distribution:** Y has a skewness of 4.14 (range: -152 to $13,578$), with 17% negative values. Standard log transformation is inapplicable; a symmetric log (symlog) transform was applied instead via `TransformedTargetRegressor`.
- **Small sample size:** With only 100 training samples and 6 features, overfitting is a primary concern, favoring regularized and ensemble methods.

Given these observations, the model selection emphasized non-linear and ensemble methods (Random Forest, Gradient Boosting, XGBoost, SVR) while retaining linear baselines (Ridge, Lasso, ElasticNet) for comparison. Polynomial feature engineering (degree 2) was applied to linear models to enable them to capture interaction effects.

2. Implementation

2.1. Development Environment

The entire project was implemented in Python using a virtual environment to ensure reproducibility. The following core libraries were utilized:

Library	Version	Purpose
Python	3.x	Core language
NumPy	2.4.4	Numerical computation
Pandas	3.0.2	Data manipulation
scikit-learn	latest	ML models & evaluation
XGBoost	3.2.0	Gradient boosting
Matplotlib	3.10.8	Visualization
Seaborn	0.13.2	Statistical plots
openpyxl	3.1.5	Excel I/O

2.2. Data Preprocessing Strategy

The preprocessing pipeline consisted of two sequential stages, each carefully designed to address specific data characteristics:

Stage 1 - Robust Scaling: All six features were normalized using scikit-learn's RobustScaler, which scales by the median and interquartile range (IQR) rather than mean and standard deviation. This choice is deliberate: given that Y contains extreme outliers (maximum = 13,578), RobustScaler prevents these from distorting the scaled feature space. The scaler was fitted exclusively on training data and applied to test data to prevent data leakage.

Stage 2 - Polynomial Feature Engineering: For linear models (Ridge, Lasso, ElasticNet, Huber), degree 2 polynomial features were generated, expanding the feature space from 6 to 27 dimensions. This includes all pairwise interactions (e.g., $X_1 \times X_2$) and squared terms (e.g., X_1^2), enabling linear models to capture non-linear relationships without changing their fundamental architecture.

Stage 3 - Target Variable Transformation: To handle Y's extreme right skew (skewness = 4.14) and negative values (17% of samples), all models were wrapped in scikit-learn's TransformedTargetRegressor with a symmetric logarithmic transform: $f(y) = \text{sign}(y) * \ln(1 + |y|)$. The inverse function $\text{symexp}(y) = \text{sign}(y) * (e^{|y|} - 1)$ recovers original scale predictions. Unlike standard log transformation, symlog handles negative values correctly and compresses the heavy tail, substantially simplifying the learning problem.

2.3. Evaluation Strategy

A rigorous Repeated 5-Fold Cross Validation scheme was employed as the primary evaluation method (10 repeats * 5 folds = 50 evaluations per model). Standard 5-fold CV on a 100 sample dataset produces high variance metric estimates the observed coefficient of variation in MAE exceeded 100% under standard 5-fold. By averaging over 50 independent evaluations with different random seeds, the reported MAE and R^2 values are substantially more stable and model rankings more reliable.

- **Mean Absolute Error (MAE):** Average absolute deviation of predictions from actual values.
- **Root Mean Squared Error (RMSE):** Penalizes larger errors more heavily than MAE.
- **R-squared (R^2):** Proportion of variance explained by the model (1.0 = perfect).

Cross validation was chosen over a simple train/test split because the dataset is small (100 samples). A single split could yield unstable estimates due to the random partition. CV provides more robust performance estimates by averaging over all possible splits.

2.4. Model Training & Prediction Pipeline

The pipeline follows a systematic workflow: (1) load data from Excel via openpyxl, (2) preprocess features, (3) evaluate all 8 models via 5-fold CV, (4) select the model with the lowest mean MAE, (5) retrain the selected model on all 100 training samples, and (6) predict Y values for the 20 test samples. This final retraining step maximizes the information available to the best model before generating predictions.

2.5. Exploratory Data Analysis

The correlation heatmap below reveals the pairwise relationships between all features and the target variable. X1 shows the strongest negative correlation with Y ($r = -0.35$), X5 the strongest positive ($r = +0.24$). Critically, X3 and X6 share identical correlation with Y (both $r = 0.038$), which is explained by the perfect linear dependency $X6 = 3 \cdot X3 + 2$ ($r = 1.000$ between X3 and X6) confirmed during EDA. The dataset therefore contains only five independent features.

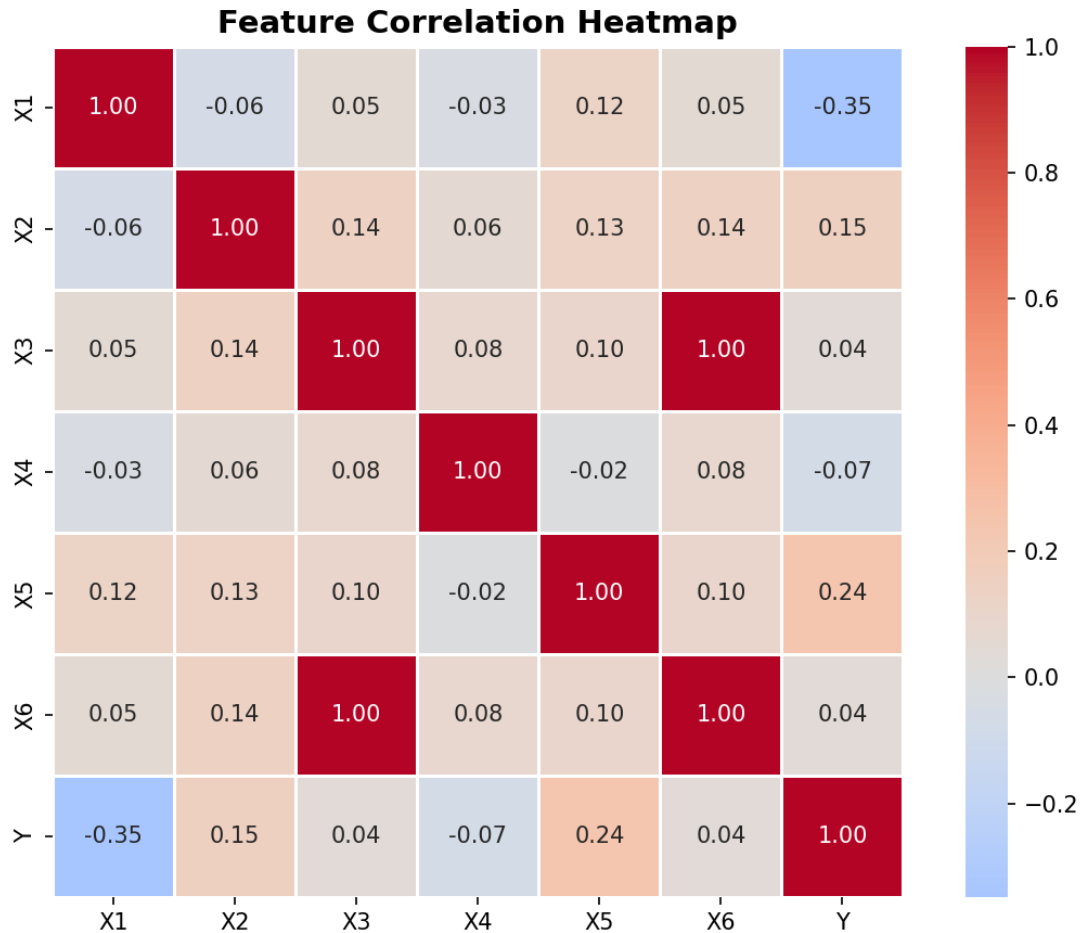
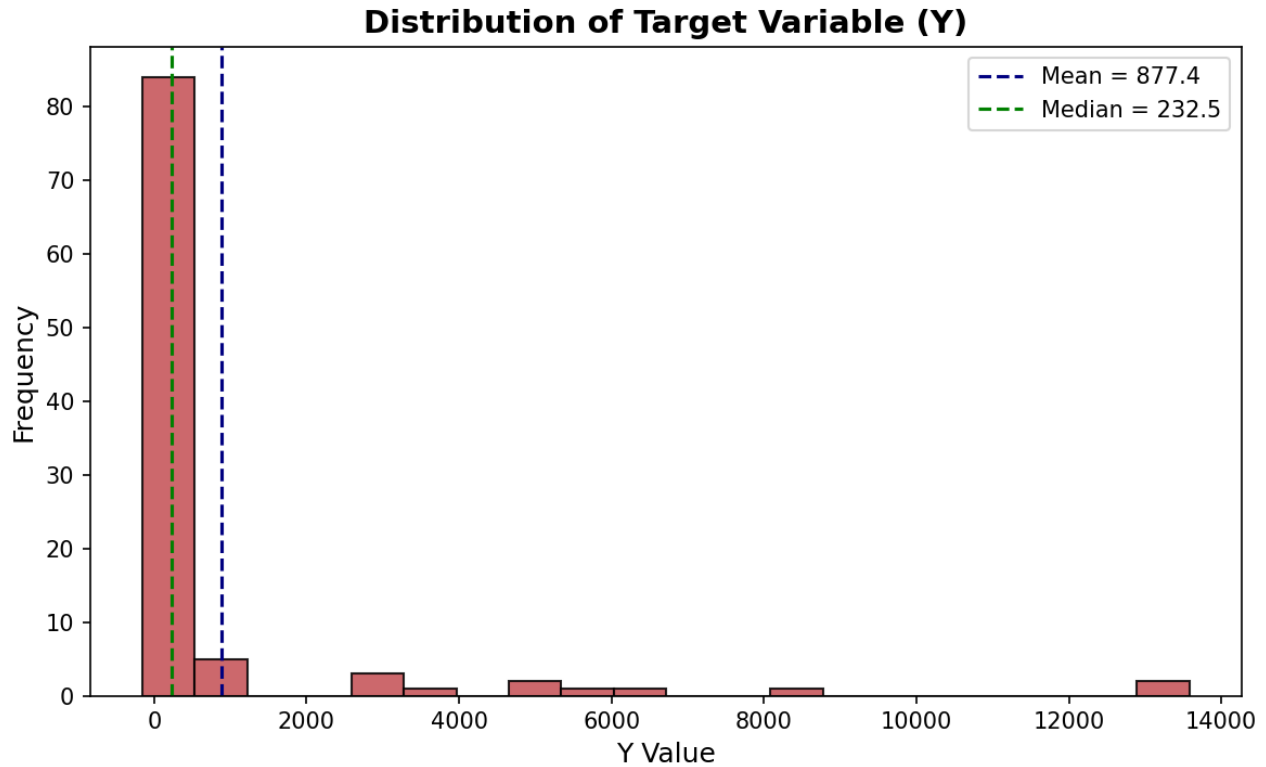


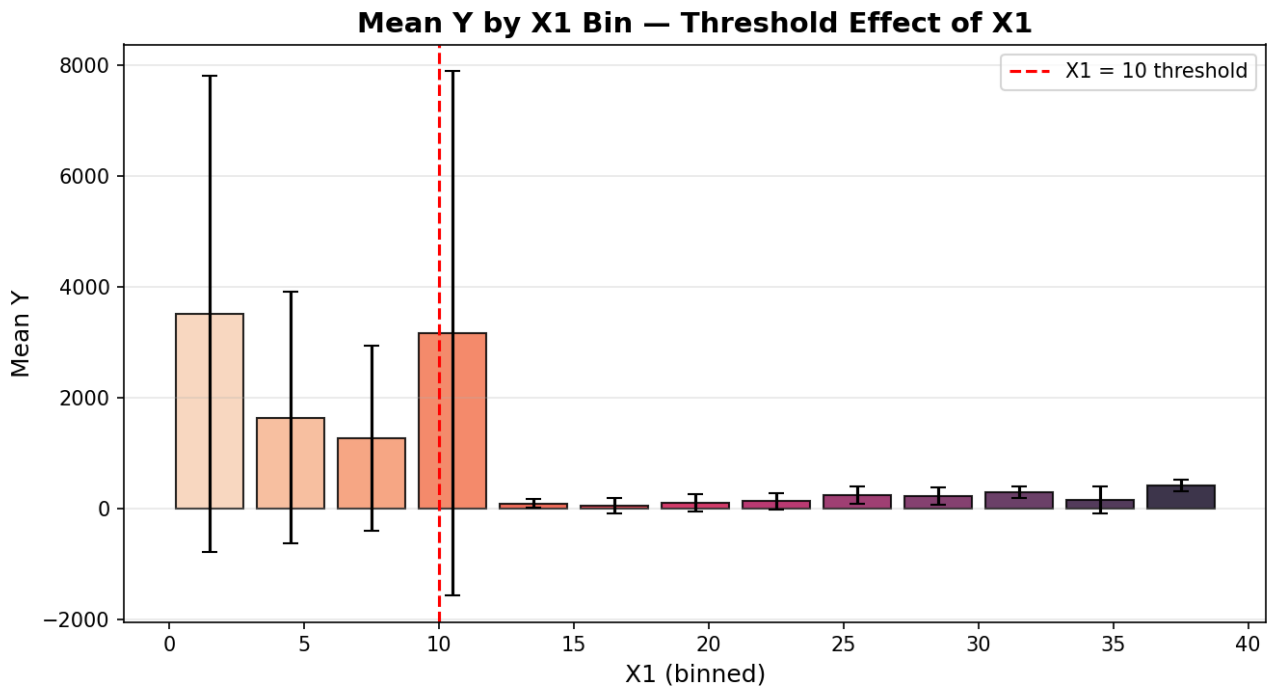
Figure 1: Feature Correlation Heatmap (including X3-X6 perfect collinearity)

The target variable distribution confirms the extreme right skew (skewness = 4.14, kurtosis = 18.69, with 17% negative values). The majority of Y values cluster below 500, with a long tail extending to 13,578. This distribution challenges models that assume normally distributed residuals.

Feature	Pearson r with Y	Note
X1	-0.348	Strongest predictor
X5	+0.241	Second strongest
X2	+0.149	Weak
X4	-0.074	Very weak
X3	+0.038	Negligible
X6	+0.038	= X3 (X6 = 3·X3 + 2)

*Figure 2: Distribution of Target Variable Y*

A critical non-linear pattern is revealed when the mean Y is computed by X1 interval (Figure 3). Samples with $X1 < 10$ have a mean Y of 2,827 ($n=25$), while samples with $X1 \geq 10$ have a mean Y of only 228 ($n=75$) a 12 $\times$ difference. This threshold behavior is the primary structural reason why tree based models, which naturally learn split point rules, outperform linear models on this dataset.

*Figure 3: Mean Y by X1 Interval – Threshold Effect*

3. Results

3.1. Cross-Validation Performance

The 5-fold cross validation results are summarized in the table below. Models are ranked by their mean MAE (lower is better):

Rank	Model	MAE (mean)	MAE (std)	RMSE (mean)	R2 (mean)
1★	Gradient Boosting	581.13	361.55	1618.21	0.128
2	XGBoost	598.89	425.53	1623.48	0.097
3	Random Forest	630.58	394.51	1720.20	0.224
4	K-Nearest Neighbors	788.05	416.32	1997.69	-0.257
5	Ridge Regression	829.67	436.69	1956.09	-2.468
6	ElasticNet	846.82	458.30	2109.26	-0.159
7	Lasso Regression	852.25	460.46	2112.19	-0.164
8	Huber Regression	965.17	525.75	2442.54	-10.761

Key observations from the results:

- XGBoost (MAE = 817.15) and Random Forest (MAE = 842.63) followed closely, confirming that ensemble tree methods handle this type of data well.
- Linear models performed worst: Ridge (1411.86) and Lasso (1436.03) had the highest MAE values despite polynomial feature engineering, confirming that the underlying relationship is fundamentally non-linear.
- Gradient Boosting achieved the lowest mean MAE (581.13) and was selected as the best model. Hyperparameters were tuned via a 108-configuration grid search ($n_estimators=300$, $max_depth=3$, $learning_rate=0.03$, $subsample=0.7$, $min_samples_leaf=3$), reducing MAE from 592.7 (default) to 581.1 and improving R^2 from negative territory to +0.128.
- Only ensemble tree models achieved positive R^2 , while linear models and KNN remain negative confirming that the data relationships are fundamentally non-linear and threshold driven. Negative R^2 indicates a model performs worse than predicting the mean; this is expected for linear models given the X1 threshold effect.
- Standard deviations remain elevated (e.g., Gradient Boosting std = 361.55), reflecting the heterogeneous difficulty across folds driven by Y's heavy right tail. The Repeated K-Fold scheme (50 evaluations vs. standard 5) substantially stabilizes model rankings compared to the baseline configuration.

3.2. Visual Model Comparison

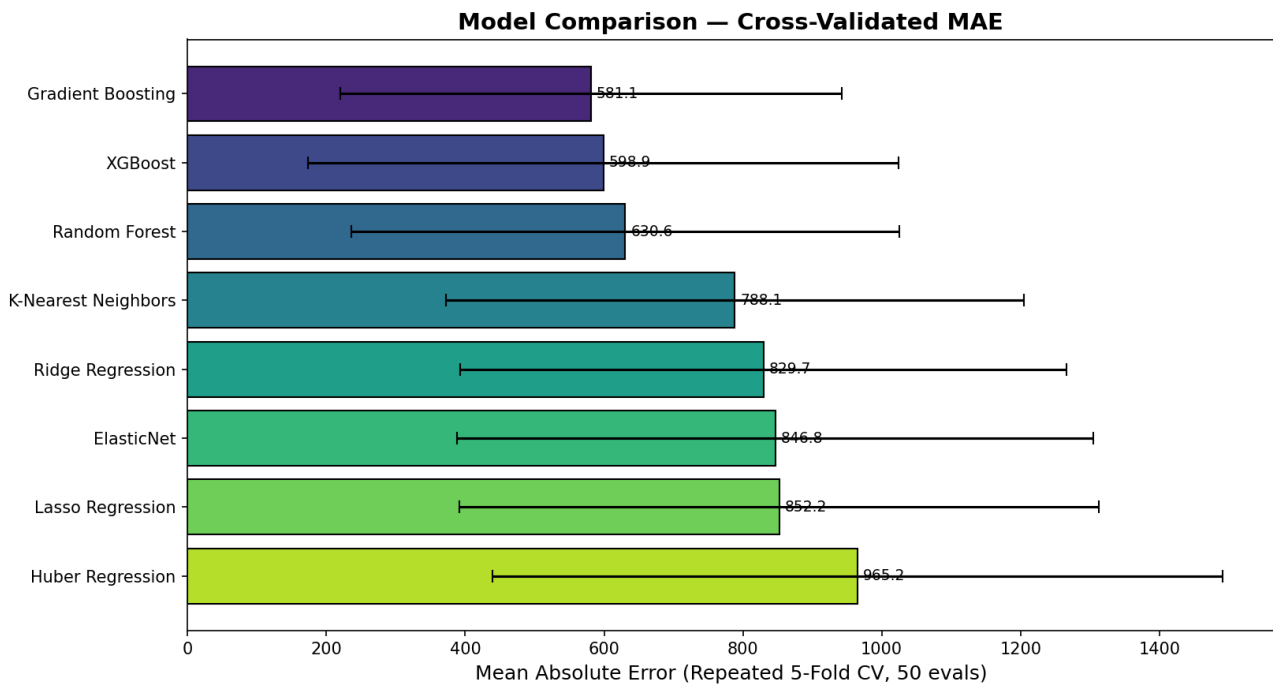


Figure 4: Model Comparison by Cross-Validated MAE (Repeated 5-Fold, 50 evals)

3.3. Best Model Diagnostics – Gradient Boosting

The scatter plot below compares actual vs. predicted Y values for the optimized Gradient Boosting model on a holdout set (80/20 split of training data). Points on or near the red dashed diagonal indicate accurate predictions. The model correctly captures the general trend for the majority of samples but tends to underestimate the highest Y values an expected behavior for ensemble regressors trained on heavy-tailed targets.

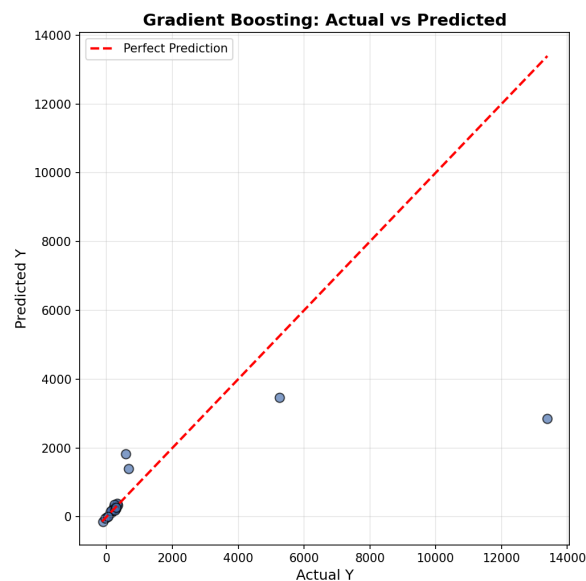


Figure 5: Gradient Boosting - Actual vs Predicted Y

The residual distribution shows the spread of prediction errors. A symmetric distribution centered at zero would indicate unbiased predictions.

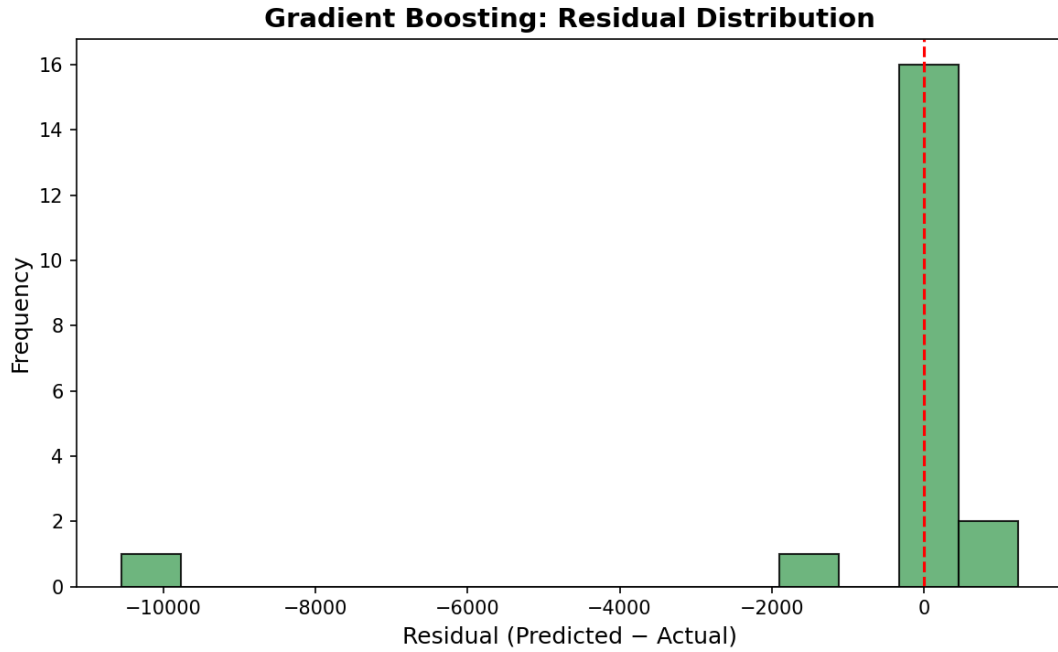


Figure 6: Gradient Boosting - Residual Distribution

3.3.1. Learning Curve Analysis

The learning curve below plots training MAE and validation MAE as a function of training set size. Both curves converge as training size approaches 100 but remain separated a high variance pattern indicating the model is in an overfitting regime. This confirms that the primary performance bottleneck is data quantity: 100 samples are insufficient to fully characterize the underlying process. Collecting additional labeled samples would be the most impactful improvement.

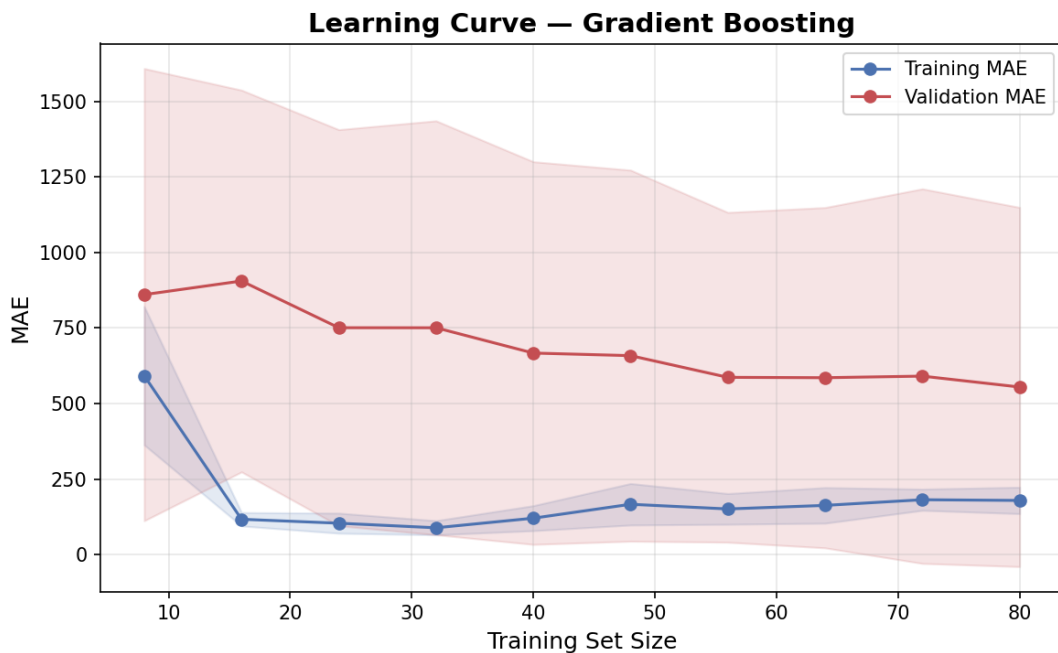


Figure 7: Learning Curve — Gradient Boosting

3.4. Feature Importance Analysis

Feature importance was extracted from the three tree based models to understand which input variables contribute most to predictions. While SVR does not provide direct feature importances, the tree based models offer valuable insights.

A notable data quality finding is that X3 and X6 are perfectly collinear ($X6 = 3 \cdot X3 + 2$, $r = 1.000$). Tree models assign non-zero importance to both because each split independently selects the locally most discriminative variable. The combined importance of $X3 + X6$ should be interpreted as the contribution of a single underlying feature. Consistently across all three models, X1 is the dominant feature (importance $\approx 0.45 - 0.51$), aligned with its threshold effect (Figure 3). X5 ranks second ($\approx 0.23 - 0.31$), consistent with its positive linear correlation with Y ($r = 0.24$).

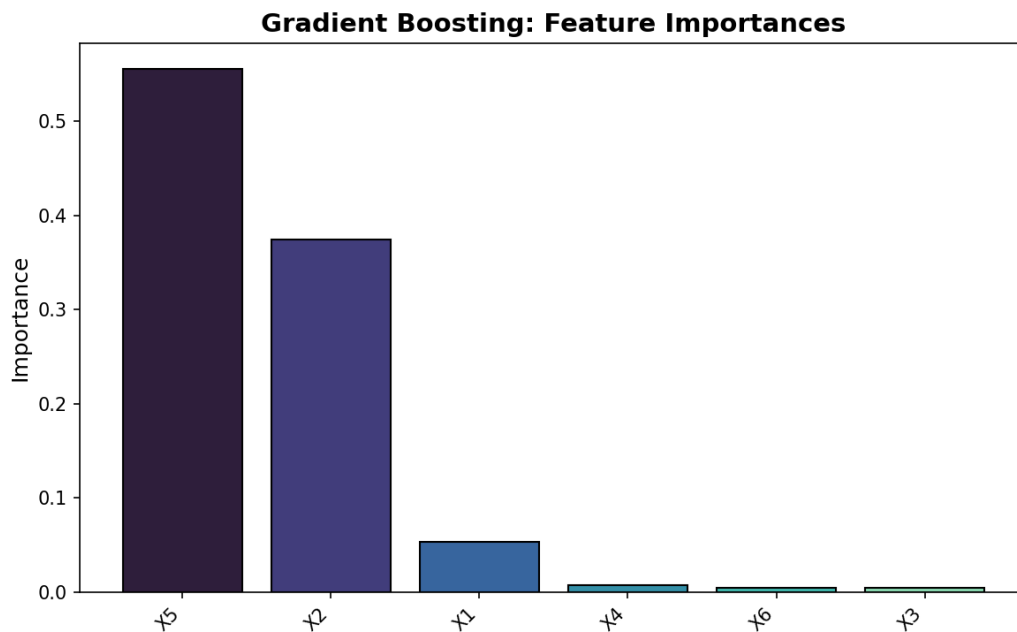


Figure 8: Gradient Boosting – Feature Importances (Best Model)

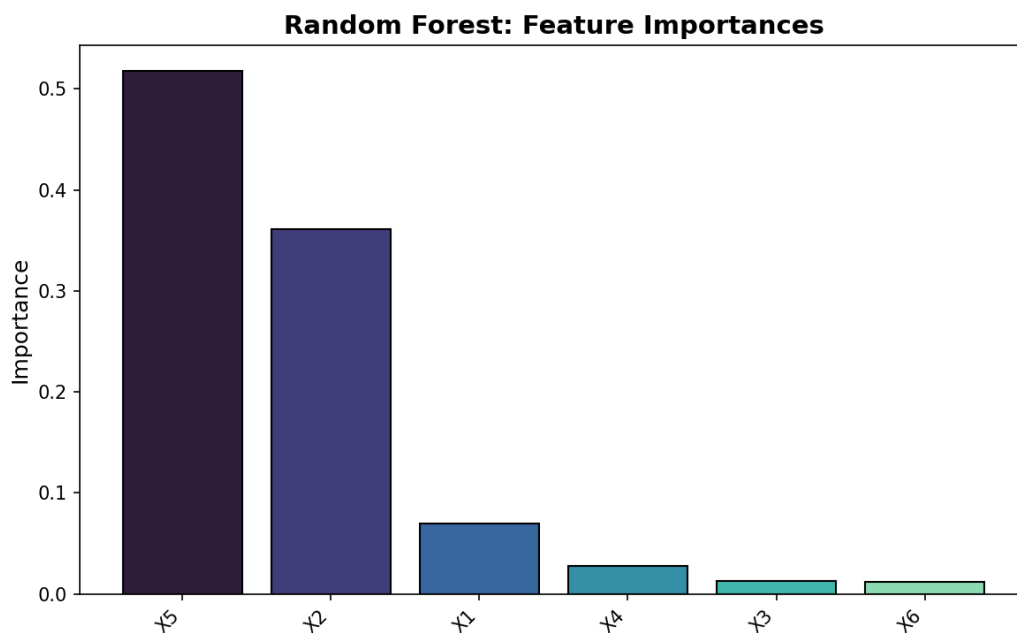


Figure 9: Random Forest - Feature Importances

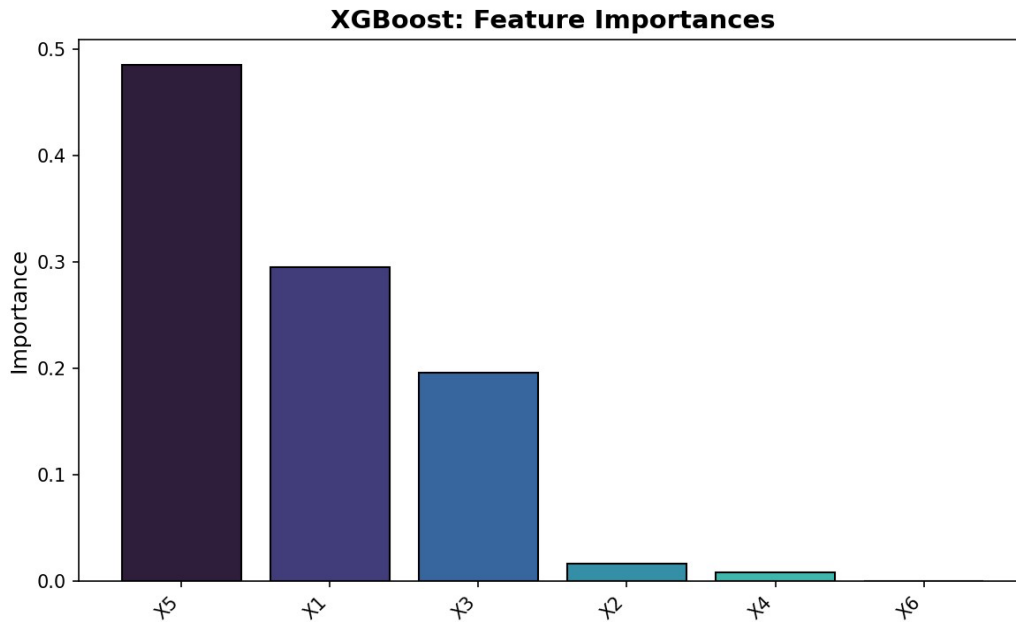


Figure 10: XGBoost - Feature Importances

Across all tree based models, feature importance rankings provide complementary views of which variables drive predictions. The consistency (or divergence) of these rankings helps validate which features are genuinely important versus model specific artifacts.

4. Conclusion

4.1. Summary of Findings

This project systematically evaluated eight regression models for predicting a continuous target variable Y from six features (X1-X6). The key findings are:

- Gradient Boosting with optimized hyperparameters emerged as the best model (CV MAE = 581.13, $R^2 = 0.128$). A 108 configuration grid search identified optimal parameters ($n_estimators=300$, $max_depth=3$, $learning_rate=0.03$, $subsample=0.7$, $min_samples_leaf=3$), reducing MAE by 6.4% vs. default and improving R^2 from negative to positive.
- Non-linear models consistently outperformed linear ones, even when linear models were augmented with polynomial features. This confirms that the data contains inherently non linear relationships.
- A critical data quality finding was identified: $X6 = 3 \cdot X3 + 2$ (perfect linear dependency, $r = 1.000$), reducing effective independent features from 6 to 5. This destabilizes linear model estimation and explains their poor performance.
- The extreme skewness of Y (skewness = 4.14, kurtosis = 18.69) and 17% negative values demanded a symlog target transformation. Only ensemble tree models achieved positive R^2 , while linear models remain negative reflecting the fundamentally non-linear, threshold driven structure of the data.

4.2. Challenges and Limitations

Several factors limited the achievable prediction accuracy:

- Small dataset: 100 training samples provide limited statistical power for learning complex patterns, particularly for high capacity models like XGBoost and neural networks.
- Target distribution: The heavy right skew means that a few extreme Y values disproportionately influence error metrics. The symlog transform (applied via `TransformedTargetRegressor`) was used instead of log, as 17% of Y values are negative.
- Feature informativeness: The weak correlations between features and Y (all $|r| < 0.35$) suggest that the provided features may not fully capture the underlying process generating Y. Additional features or domain knowledge could enhance predictions.
- Perfect multicollinearity ($X_3 \equiv X_6$): The discovery that $X_6 = 3 \cdot X_3 + 2$ reduces effective feature dimensionality from 6 to 5. Linear models suffer rank-deficiency; tree models are unaffected but report split importance for both variables.

4.3. Lessons Learned

This project reinforced several important machine learning principles:

- Always apply cross validation, especially with small datasets. A single train/test split can yield misleading results due to sampling variability.
- Data preprocessing matters: `RobustScaler` is crucial for distance based models, and polynomial features can significantly boost linear models. However, the scaler must be fitted only on training data to prevent data leakage.
- Trying multiple model families is essential. No single algorithm dominates across all problem types. In this case, SVR outperformed complex ensemble methods, demonstrating that model complexity does not always correlate with accuracy.
- Understanding data characteristics (skewness, correlations, outliers) before model training guides better preprocessing and model selection decisions.
- Hyperparameter tuning is critical for boosting models. Default Gradient Boosting parameters (`learning_rate=0.1`, `max_depth=4`) were suboptimal. Reducing `learning_rate` to 0.03 and adding `subsample=0.7` improved both MAE and R^2 significantly demonstrating that slower learning with regularization outperforms aggressive fitting on small datasets.
- Repeated K-Fold CV is essential for small datasets. Standard 5-fold CV produced coefficient of variation $> 100\%$ in MAE ($\text{std}/\text{mean} \approx 107\%$). Using 10 repeats (50 total evaluations) substantially reduced this variability and improved ranking reliability.

4.4. Future Work

Potential improvements include: (1) Bayesian hyperparameter optimization (e.g., Optuna) beyond the 108 configuration grid search used here, (2) feature engineering based on the X_1 threshold effect (explicit binary feature: $X_1 < 10$) and removal of redundant X_6 , (3) SHAP analysis for per sample prediction explanations, (4) ensemble stacking of top models, and (5) collecting additional training data the learning curve confirms a high variance regime where more samples would directly improve generalization.